COMS3011A Project Brief

Brendan Griffiths

Contents

1 Overview 2
1.1 Projects & Groups 2
1.2 Milestones & Reports 2
1.2.1 Milestone 1: Sprint 1 2
1.2.2 Milestone 2: Sprint 2 2
1.2.3 Milestone 3: Sprint 3 2
1.2.4 Milestone 4: Submission 2
1.2.5 Report 1: Group Report 3
1.2.6 Report 2: Individual Report 3
1.2.7 Report 3: Peer Review 3
1.2.8 Report 4: Group Presentation 3
1.3 Timeline 4
1.4 Artificial Intelligence 4
2 Projects 4
2.1 Key Requirements of All projects 4
2.2 Project 1: Tutor Management System 5
2.3 Project 2: Sport Coaching Tool 5
2.4 Project 3: Sport Analytics Tool 6
2.5 Project 4: Automata Editor and Visualiser 6
2.6 Project 5: Olympiad Portal 6
2.7 Project 6: Campus Game - Wits Quest 6
2.8 Project 7: Digital Logbook 6
2.9 Student Project 7
3 Rubrics 7
3.1 Milestone 1: Sprint 1 7
3.2 Milestone 2: Sprint 2 8
3.3 Milestone 3: Sprint 3 10
3.4 Milestone 4: Project Submission 11
3.5 Report 1: Group Report 12
3.6 Report 2: Individual Report 13
3.7 Report 3: Peer Review 14
3.8 Report 4: Group Presentation 15

1 Overview

1.1 Projects & Groups

Each project is broken down into three tiers: Basic, Intermediate, Advanced. To achieve the basic tier, your project needs to meet all the listed key and basic requirements, as well as achieve the standard of a *minimum viable product*. The intermediate tier needs to meet all the listed basic requirements and the intermediate requirements. The expected standard is that of a functional and useful product. The advanced tier needs to meet *all* listed requirements, even those potentially introduced part way through the semester. The standard of an advanced project is one that can be released to market the day of submission.

You can find a brief problem statement for each project in this document, but additional information will be provided through project specific briefs.

Groups are 4 - 6 members. Please select your group carefully as you may not change your group once the semester has started.

Your group will be assigned a tutor as a 'client'. Their role is to help guide you on the development process and make decisions around if a feature is 'complete' or not. Your tutor will act as the first point of contact for any questions about your project, but the lecturer will be available if you would like further clarification. You will *not* be marked by your client, but by other tutors. The client is however expected to provide some context to the marking tutor on your interactions with them.

1.2 Milestones & Reports

1.2.1 Milestone 1: Sprint 1

Milestone 1 is aimed at project setup and the basics of a project. Setting up the project repositories, getting the largest 'blockers' out of the way, making sure you and your group have a strong direction for the rest of the semester.

1.2.2 Milestone 2: Sprint 2

Milestone 2 is the midpoint of the project, where by the end of this sprint you have a strong base for the project. The most important features are implemented, even if there are issues, and the project is moving towards a stable product through testing and reviews.

1.2.3 Milestone 3: Sprint 3

Milestone 3 is aimed at getting the project into a near-complete state. Most features should be implemented, and mostly working. The project should have gone through reviews and you should be evaluating the state of the project from a qualitative perspective.

1.2.4 Milestone 4: Submission

Milestone 4 is the final submission. Your project should work, work well, and do what it needs to.

The marking of milestone 4 is evaluating your entire project holistically. Over the entire dev-cycle of the project, has your group been consistent with your methodology, how has your projects requirements and plans evolved.

1.2.5 Report 1: Group Report

The group report is a summary of the project's development and a reflection as a group of the entire project. What decisions were made through out the project, why did they occur and what can be done to improve upon them in the future.

The report should be presented as a technical document with references, an introduction, body, conclusion and appendices.

1.2.6 Report 2: Individual Report

The individual report is a reflection of your growth and development throughout the project. What challenges did you encounter, what new technology did you learn about, how did you work as a team, and most importantly, something you are proud of.

The report is an opinionated piece, but should still be presented as academic writing. It should have an introduction, body, conclusion, appendices and use references and evidence based motivation.

1.2.7 Report 3: Peer Review

The peer review is split into two sections:

1. An anonymous review of your group members
2. A personal evaluation of your performance

Your mark is calculated by taking your personal rating and the average rating of your group members.

The peer review will act as a multiplier of your group marks. If you achieve a peer review mark of greater or equal to 50%, you will receive all your group marks. If you achieve a mark less than 50%, you will receive a mark multiplied by a specified value for every group component of the project.

Mark Range	Group Component Multiplier
[50%, 100%]	$\times 1$
[40%, 50%)	$\times 0.8$
[25%, 40%)	$\times 0.6$
[15%, 25%)	$\times 0.4$
[0%, 15%)	$\times 0.2$

1.2.8 Report 4: Group Presentation

The presentation is to demonstrate you understand your project at a technical level, and to 'pitch' it. You must be able to talk about your project, the decisions you made, what makes it cool, and why is it good. The presentation is a great opportunity to bring attention to specific features that might otherwise be overlooked.

You will present your product to a panel of judges, and must cover the following information:

- an overview of your product
- feature demonstrations
- UI and UX showcase
- The product's architecture and features at a high level (no code)

1.3 Timeline

The timeline of your project is over the entire semester. The below dates are preliminary and may be adjusted over the semester.

Event	End Date	Days from Previous
Group Selection	2026-07-26	
Student Project Proposal	2026-07-31	5
Project Assignment	2026-08-02	2
Milestone 1: Sprint 1	2026-08-25	23
Milestone 2: Sprint 2	2026-09-15	21
Milestone 3: Sprint 3	2026-09-29	14
Milestone 4: Submission	2026-10-11	12
Report 1: Group Report	2026-10-18	7
Report 4: Group Presentation	2026-10-19 ⟶ 2026-10-23	5
Report 2: Individual Report	2026-10-27	4
Report 3: Peer Review	2026-10-27	0

There are 70 days of dedicated development time for the project.

The back half of this semester will coincide with the CGV project. Please make sure that you have adequately prepared for the reduction in your available development time.

1.4 Artificial Intelligence

For the duration of this project, you are very encouraged to use AI tooling. To encourage transparency and ethical usage, you must attribute your usage of AI.

For complete details of how to attribute AI, please review the dedicated AI policy document, but as a brief summary, you must specify what tool (Claude Code, ChatGPT Web), what model (GPT-5.5, Claude Opus 4.8) and purpose (code, editing).

You are responsible for everything generated and used by AI during this project. Any hallucinations and incorrect artifacts *will* impact your marks. You should be aware that usage of AI may impact the subjective components of marking as it may insufficiently demonstrate or answer the question with the required depth.

2 Projects

2.1 Key Requirements of All projects

Every project needs to implement following requirements.

- Version Control: You are expected to be using some form of git-compliant version control software

- Responsiveness and Accessibility: Your project should conform with modern standards of web applications when it comes to their responsiveness and accessibility.
- CI / CD: Your project should make use of CI/CD processes for both the collaboration and deployment of the application
- A non-monolithic front-end and back-end application
  - If a framework provides a way to implement your front-end and back-end code together, you may use it for either front-end or back-end, not both.
  - Some examples: Sveltekit, Next.js
  - If you are in doubt, reach out to the lecturer and ask.
- Hand-Written API
  - You must design and implement your API
  - You may not use a system that generates API endpoints
    - Unless you write the generation as part of your project
    - examples: Firebase, Supabase
  - If in doubt, you should be making requests through an HTTP endpoint
  - If you are unsure about a technology, speak to the lecturer for clarification.
- Authentication & Security
  - Users must be able to sign up, sign in, reset their passwords, and delete their accounts
  - You may not write your own authentication system, instead you must rely on established practices and libraries.
- Integration
  - You must integrate with a *relevant* external API service in some manner.
  - For example: A hiking app checks the weather of the hike's location
- Documentation Website
  - Your project must have a publicly available website that has your documentation
  - Avoid platforms that require an account such as Notion.
  - As a suggestion, use a pre-established docs tool built on top of markdown
    - Docusaurus, MkDocs, mdBook
  - Deploy this site via static web-hosting
    - Github Pages, Cloudflare Pages

2.2 Project 1: Tutor Management System

The school of Computer Science and Applied Mathematics is trying to come up with a better way to manage their tutors. They need a platform that allows an organiser to assign the various available tutors to different courses based on a few criteria: The tutor's marks in the course, their availability with their own timetable, and hours per week. This platform also needs to provide a way for students to volunteer for any overflow work, sign and submit their timesheets, excuse themselves and more admin capabilities.

2.3 Project 2: Sport Coaching Tool

A local coach for (insert favourite sport here) has recently complained that they don't have any good tools to help them manage their sport. They want to be able to use one tool that allows them to track events, player statistics, compare with other teams, track any history and so on. They should be able to

organise and plan events with other users of the platform, and have assistants (or themselves) track event history (results, penalties etc.)

2.4 Project 3: Sport Analytics Tool

Large leagues often have databases to help track an athlete's performance. Unsatisfied with (insert favourite sport here)'s current offerings, you have decided to build one of your own for a professional league. This platform needs to allow approved users to submit and manage the statistics for players (or teams), and provide an API for other platforms to use. All statistics that are important for (sport) should be available, and preferably calculated from event data instead of manual entry. This platform should serve as a home for anyone who wants to get into big-data analysis of the sport and should offer API's and datasets to help this process.

2.5 Project 4: Automata Editor and Visualiser

Your FLA lecturer has approached with an idea to help students learn Turing Machines. They would like to build an editor and visualiser for Turing Machines that allows students to edit and save the various machines. The editor needs to be a visual editor of the state diagram, and a code editor for the instructions. The visualiser needs to show and highlight the current configuration of the machine for a specified input and track useful information about the computation.

2.6 Project 5: Olympiad Portal

An organiser for a local olympiad wants to build a portal for competing schools and educators to register and upload results on. The portal should allow educators to download tests and past papers, submit results and have them automarked if it is a multiple choice questionnaire. They would prefer if the portal also provides a way for students to take the test online if possible, but the offline tests should always be available.

2.7 Project 6: Campus Game - Wits Quest

Inspired by Pokemon Go and tired of the current tours, a lecturer wants to build a web game that gets students (and anyone else) to explore campus. When a player is close to an event, they can participate in a game based off alumni, history, landmarks and any other trivia about the university. If a player is correct or wins the game, they are awarded collectible cards with attributes and properties that can be used in a turn based poke-battle-esque minigame with other players or CPUs.

2.8 Project 7: Digital Logbook

A uni student would like a system that can help them keep track of their projects, what work they've done and what they still need to do. They've been relying on a physical logbook to help them achieve this, but they always forget it at home. To fix this, they've asked you to help build a digital logbook that they can customise the format of, and then quickly capture entries for. This logbook needs to give them useful statistics about what they are doing, how much time they've spent on specific projects. Most importantly however, it needs to give them a record of what they do.

2.9 Student Project

If you have your own idea, you may submit a proposal document that gives us an outline of the project. This document needs to have the following:

- A problem statement - what issue is this project going to solve
- An explanation of how the project meets all the key requirements
- A detailed feature list that categorises features into the three tiers: basic, intermediate and advanced
- Group motivation of why you want to build this project over any of the listed projects
- Significant planning of how you are going to implement the most important features

If you submit your own project, there are a few things you **must** agree to:

- The lecturer reserves the right to reject your proposal where you must then choose one of the listed projects
- Every group gets only one proposal, if it is rejected, you may not submit another
  - You may appeal this decision during a lab where we can discuss in more detail
- You will be held to the standard of your initial proposal
  - Ambition is good, but if you are overly ambitious in the proposal and then fail to reach it, you will be penalised
  - A proposal that does not meet the minimum required standards of the course will be rejected
- A tutor will still be assigned as a 'client' with a slightly modified role
  - The tutor will not make final decisions on which features are in the project
  - The tutor will help build a sustainable development timeline for the project, even if that means they suggest you cut specific features
  - The tutor is there to help you meet the basic requirements you outline in your project proposal

3 Rubrics

3.1 Milestone 1: Sprint 1

Criteria	**Weight**	**Advanced: 80% - 100%**	**Intermediate: 50% - 80%**	**Basic: 0% - 50%**
Version Control	10%	Exists - code quality tools, commit structure Used - All members have committed code	Exists - organised file structure & readme, Used - Most members have committed code	Exists - no repo → basic organisation Used - Some members have committed code
Documentation Site	10%	Well-designed, has non-trivial content	Has trivial content	No documentation, Local documentation, online but

Criteria	Weight	Advanced: 80% - 100%	Intermediate: 50% - 80%	Basic: 0% - 50%
				inaccessible, version controlled
Getting Started / Dev Guides	5%	All repo's feature a README and getting started	Some to Most repo's have a README	No to Some repo's have a README
Work Tracker	5%	Exists & Used	Exists	Does not exist
Git Methodology	5%	Used	Documented	No methodology → insufficient methodology
Project Methodology	10%	Used methodology with evidence, methodology is motivated	Methodology is documented and referenced	No methodology, invented methodology, listed but unexplained methodology
Tech Stack	5%	Fully listed stack, motivated	Most stack components listed and motivated	None to Some stack components listed
Stakeholder Interaction	10%	Great stakeholder interaction, all requirements discussed	Identified most requirements with discussion	None to poor stakeholder interaction and requirement identification
Initial Design & Dev Plan	20%	Comprehensive dev plan, system design, ui design, roadmap	Adequate plan	Incomplete, partial plan
Implementation	20%	All supporting requirements and features implemented	Most supporting requirements and features implemented	None to some supporting requirements and features implemented

3.2 Milestone 2: Sprint 2

Criteria	Weight	Advanced	Intermediate	Basic
Core Features	25%	Implemented with at most one non-severe bugs	Implemented with some non-severe bugs, or one severe bug	Not fully implemented or featuring severe bugs

Criteria	Weight	Advanced	Intermediate	Basic
Automated Testing	10%	UI and API testing implemented - useful and extensive tests	UI or API testing with simple or default tests	No testing
Stakeholder Reviews	10%	Extensive evidence of regular stakeholder interaction, evaluated and integrated stakeholder feedback	Evidence of stakeholder interaction, evaluated stakeholder feedback	No stakeholder feedback, stakeholder feedback ignored without motivation
API	15%	API available externally with documentation and have integrated external API	Mostly implemented API, some external integration	No / minimal external API integration, API not available
User Feedback	10%	Extensive User testing - Formal Feedback collection processes, evidence of collection and integration	Some user testing - formal feedback collection process, evidence of collection, but no integration	No user testing, Non-formal collection
Project Methodology	10%	Actively following methodology	Mostly following methodology	None to poor following of methodology
Bug Tracker	5%	Extensive and continuous use of bug tracker	Use of a bug tracker	No use of a bug tracker
Database Documentation	5%	Full schema documentation, deployment info and motivation	Some db documentation, deployment info	No documentation
Third-Party Code Documentation	5%	All third-party code documented and motivated	Most third party code documented	No to poor documentation of third party code
Testing Documentation	5%	Extensive documentation on testing - user feedback formal process, automated	Some testing documentation - user feedback process, automated testing	None to poor documentation

Criteria	Weight	Advanced	Intermediate	Basic
		testing procedure, policy around tests		

3.3 Milestone 3: Sprint 3

Criteria	Weight	Advanced	Intermediate	Basic
User Feedback	10%	Extensive user testing and feedback gathering	Some user testing and feedback gathering	No user testing
Automated Testing	10%	Extensive automated testing across all code bases (code coverage $>$ 60%)	Extensive automated testing in one code base / Middling testing across all code bases (code coverage $>$ 30%)	No automated testing / low automated testing / middling testing in one code base
Feature Implementation	20%	Most features implemented with minimal bugs / All features implemented with non-severe bugs	Most features implemenented with non-severe bugs / All features implemented with bugs	No additional features from sprint 2 / some additional features with bugs / most features implemented with bugs
API Implementation	20%	All endpoints implemented mostly bug free	All endpoints implemented with non-severe bugs	Not available for external usage / Some endpoints implements / most endpoints implemented with bugs
Performance	5%	No performance issues	Minimal performance issues	Large performance issues
Improvement	5%	Extensive evidence of integrating user and stakeholder feedback	Some evidence of integrating user and stakeholder feedback	No evidence of integrating user and stakeholder feedback
Documentation	15%	All aspects (features / API / database / testing et.c) of the project are *well* documented	Most aspects of the project are *well* documented	No aspects of the project are documented / Some aspects of the project are documented / Most

Criteria	Weight	Advanced	Intermediate	Basic
				aspects of the project documented
Project Methodology	15%	Evidence of strictly following methodology	Evidence of mostly following methodology	No or poor evidence of following methodology

3.4 Milestone 4: Project Submission

Criteria	Category	Weight	Description
Data	Database	3%	Do you have production data? How much data is from testing?
Deployment	Database	2%	Is your database deployed and available?
Structure	Database	5%	How well structured is your database? Are you following the best practices for your database type?
Availability	API	3%	Is your API available for external and internal usage?
Architecture	API	5%	How well does your API follow an established API architecture and all of its principles?
Deployment	API	2%	Is your API deployed?
Performance	API	5%	How well does your API handle significant load? How often does it crash? How long does it take to process the first request?
Design	API	10%	Does your API makes use of the different HTTP methods correctly? Do you have redundant endpoints? Does the organisation and heirarchy make sense to an external user?
Accessibility	App	5%	Is you product accessible? Does it comply with best practice on elements such as Contrast Ratio, use of semantic HTML or ARIA?
Aesthetics	App	5%	Does your product look good? Does it have consistent styling? Is it generic or have you made an effort on the style?
User Experience	App	5%	How do you handle errors? How long does it take a user to use some feature? Can the user re-open a closed tab and continue where they left off or do they need to re-login?
Deployment	App	2%	Is you application deployed?
Performance	App	5%	How long does it take to load? How responsive is the program to input?
Features	App	10%	How well implemented are your features? Does your product do what your client and original scope intended it to?

Criteria	Category	Weight	Description
Responsiveness	App	5%	How responsive is your UI? Does it work (or give a warning) on mobile? Does it change according to screen size?
Structure	App	5%	How complex is the interface? How difficult is it to navigate to a new feature? Does it make sense to a first time user?
Git Methodology	Misc	5%	How well did you follow your chosen git methodology?
Integration	Misc	3%	Are you integrating with an external API? Is the integration cohesive with the application or bolted on?
Testing	Misc	10%	How well written are your tests? What is your code coverage ($> 80\%$? $> 50\%$)? What are you not testing?
Tools	Misc	5%	Did you use a project tracker? Did you use a bug tracker? What code quality tools did you use and how did you enforce them?

3.5 Report 1: Group Report

The group report is a technical analysis of the project. For each section, you should be looking at your documentation over the semester and identifying the motivation behind decisions, discussing how these decision played out over the semester, and ultimately if they were successful or not. You need to be substantiating your argument with evidence and references. This report should be aimed at another engineer who is trying to learn and understand the group's processes

In the below rubric, each criteria should be a standalone section of the report that covers the relevant items in the description. The introduction should serve as a summary of what can be found in the overall report and relevant sections. The conclusion should act as a final closing argument, summarising each section with its analysis, focusing on the most important aspects of each section. The final appendix should reference or be snapshots of your original documentation (not import the whole site).

Importantly, you are marked on your analysis of the project, not the project itself. If something failed, or was a continuous problem, you need to analyse why. A good analysis will receive good marks, even if the target of it was 'bad'. You should view this as a collective post-mortem on the project, and we are assessing your ability to understand and grow as developers.

Criteria	Mark	Description
Introduction	10	
Methodology & Planning	30	What was your development methodology, and did it work? How does your original project design align with your current design, and why it has (or not) drifted? How useful was your original project planning, and what changes did you make?

Criteria	Mark	Description
		How successful and viable was your stakeholder interaction? Did it help build a better project?
Architecture	20	What is the architecture of your project? How did the API develop? How was the database, and what issues did you encounter? How well were developers able to onboard and make new changes? What parts of your tech stack changed?
Testing	20	How did your automated testing procedure develop? How did you collect user feedback, and did you need to change it? Did the user feedback provide useful changes, and was it an effective part of the dev cycle?
Deployment & Integration	20	How did the deployment platform evolve? What was your CI/CD process? What external APIs did you use, and how useful was the integration?
General Discussion	20	What challenges were encountered in development? What is the future of this project? What is your opinion as a group on the project?
Conclusion	5	
Supporting Documents / Appendix	5	An appendix of your documentation and evidence you use to substantiate your arguments in the report.
Total	130	

3.6 Report 2: Individual Report

The individual report is a personal technical analysis of the project. Compared to the group report, you should be discussing the parts of the project you had an influence over, and your own learning. You need to be substantiating your analysis with evidence and artefacts to support your argument. The report should be aimed at another engineer who is evaluating you, your contributions, and your growth.

In the below rubric, each criteria should be a standalone section of the report that covers the relevant items in the description. The introduction should serve as a summary of what can be found in the overall report and relevant sections. The conclusion should act as a final closing argument, summarising each section with its analysis, and provide reflection on what you would do differently going forward. The final appendix should reference or be snapshots of your original documentation (not import the whole site).

You are marked on your analysis of the project, not the project itself. If something failed, or was a continuous problem, you need to analyse why. A good analysis will receive good marks, even if the target of it was ‘bad’. You should view this as a personal post-mortem on the project, and we are assessing your ability to understand and grow as developers.

Criteria	Mark	Description
Introduction	10	
Responsibilities & Contributions	20	What elements were you responsible for? How successful were your contributions? What role do you think you took in the development of the project? What was your impact in the overall project? Do you think you could have contributed more or in other places?
Technical Challenges & Solutions	20	What technical problems did *you* encounter? How might you pre-empt these problems in the future? What was the worst technical decision you made in the project? What was the best technical decision you made in the project?
Learning & Development	20	How did the use of AI (and what AI) affect your development of the project? Has your opinion on AI usage in software changed, and why? What other new-to-you technology did you use in the project, and how did you learn it? What technology do you *think* might have been useful, but did not use?
Collaboration & Teamwork	10	What is your opinion on the collaborative software development process you used? How has the project changed your perspective on working with other engineers? What social collaboration techniques did you use (e.g. blind-voting, brainstorming etc)?
General Discussion	10	What is something that you are proud of, or would like to redo? What is your general opinion (and why) on the project?
Conclusion	10	
Supporting Documents / Appendix	5	An appendix with your documentation and evidence you use to substantiate your arguments in the report.
Total	100	

3.7 Report 3: Peer Review

For each member of your group, you must write a brief paragraph (200 - 300 words) discussing each group members contribution. Then, you must provide a rating from 0 to 5 for each member. You will also provide a rating for yourself, out of 5, that contributes to your final mark.

Your peer review mark is calculated as the average of each rating on you, from your members over 5, and your own rating.

$$0.05 \times x_j + 0.95 \times \frac{1}{n-1} \sum x_i$$

where n is the group size and x_i is the rating from the i-th group member and x_j is your own rating.

Please see the below table for an example of a five person group.

	Person 1	Person 2	Person 3	Person 4	Person 5
Person 1	5	4	4	3	5
Person 2	1	5	5	4	2
Person 3	2	4	5	5	3
Person 4	4	4	2	5	4
Person 5	5	4	3	5	5
Mark	3.1	4.05	3.575	4.2875	3.575
%	62%	81%	71.5%	85.75%	71.5%

3.8 Report 4: Group Presentation

Criteria	**Mark**	**Description**
Product Showcase	40	Product Overview Feature Demonstration User Interface and Experience
Technical Breakdown	20	Architecture Overview Functionality Implementation
Presentation Quality	40	Structure, Time Usage, Product / Project Coverage, Visual Aids, Team Collaboration
Questions	10	How well do you respond to questions (understanding and answer) Team collaboration on question responses
Total	110	

AI Declaration: Claude Web (Sonnet 5.5) - Reviewed